

=== ИНСТРУКЦИЯ ДЛЯ НЕЙРОСЕТИ / AI INSTRUCTION ===

Внимание! Данный PDF-файл содержит исходный код проекта. Из-за ограничений формата PDF все графические эмодзи в коде были автоматически заменены на текстовые теги двоеточиями (например, смайлик зеленой галочки заменен на :check_mark_button:, крестик на :cross_mark: и т.д.).

ОБЯЗАТЕЛЬНОЕ ТРЕБОВАНИЕ: Когда ты будешь анализировать этот код, исправлять его или генерировать новые участки кода по запросу пользователя, автоматически преобразуй все текстовые теги эмодзи обратно в их реальные графические Unicode-символы (смайлики)! В возвращаемом коде не должно быть текстовых тегов, только настоящие эмодзи, чтобы код корректно работал на сайте.

=====

Название файла с расширением: .gitignore

/build

Название файла с расширением: build.gradle.kts

```
plugins {
    alias(libs.plugins.android.application)
//id("com.android.application")
    id("com.google.gms.google-services")
}

android {
    namespace = "com.example.namahatta"
    compileSdk {
        version = release(36) {
            minorApiLevel = 1
        }
    }
}

defaultConfig {
    applicationId = "com.example.namahatta"
    minSdk = 24
    targetSdk = 36
    versionCode = 1
    versionName = "1.0"

    testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"
}

buildTypes {
    release {
        isMinifyEnabled = false
        proguardFiles(
            getDefaultProguardFile("proguard-android-optimize.txt"),
            "proguard-rules.pro"
        )
    }
}

compileOptions {
```

```

        sourceCompatibility = JavaVersion.VERSION_11
        targetCompatibility = JavaVersion.VERSION_11
    }
}

dependencies {
    implementation("androidx.core:core-ktx:1.12.0")
    implementation("androidx.appcompat:appcompat:1.6.1")
    implementation("com.google.android.material:material:1.11.0")
    implementation("androidx.activity:activity:1.8.0")
    implementation("androidx.constraintlayout:constraintlayout:2.1.4")

    // Firebase — через BOM (правильно!)
    implementation(platform("com.google.firebase:firebase-bom:34.9.0"))
    implementation("com.google.firebase:firebase-messaging") // ← без версии!
    implementation("com.google.firebase:firebase-analytics")

    // OkHttp
    implementation("com.squareup.okhttp3:okhttp:4.12.0")

    // JSON
    implementation("org.json:json:20231013")

    testImplementation("junit:junit:4.13.2")
    androidTestImplementation("androidx.test.ext:junit:1.1.5")
    androidTestImplementation("androidx.test.espresso:espresso-core:3.5.1")
}

```

Название файла с расширением: google-services.json

```

{
  "project_info": {
    "project_number": "55071412526",
    "project_id": "namahata-notifications",
    "storage_bucket": "namahata-notifications.firebaseio.com"
  },
  "client": [
    {
      "client_info": {
        "mobilesdk_app_id": "1:55071412526:android:a5c678a95d19463eb32abf",
        "android_client_info": {
          "package_name": "com.example.namahatta"
        }
      },
      "oauth_client": [],
      "api_key": [
        {
          "current_key": "AlzaSyBixfGmqf0du0jwEEVZK0uR2txe1vNnH04"
        }
      ],
      "services": {
        "appinvite_service": {
          "other_platform_oauth_client": []
        }
      }
    }
  ]
}

```

```

    }
  }
},
"configuration_version": "1"
}

```

Название файла с расширением: proguard-rules.pro

```

# Add project specific ProGuard rules here.
# You can control the set of applied configuration files using the
# proguardFiles setting in build.gradle.
#
# For more details, see
# http://developer.android.com/guide/developing/tools/proguard.html

# If your project uses WebView with JS, uncomment the following
# and specify the fully qualified class name to the JavaScript interface
# class:
#-keepclassmembers class fqcn.of.javascript.interface.for.webview {
#   public *;
#}

# Uncomment this to preserve the line number information for
# debugging stack traces.
#-keepattributes SourceFile,LineNumberTable

# If you keep the line number information, uncomment this to
# hide the original source file name.
#-renamesourcefileattribute SourceFile

```

Название файла с расширением: release\output-metadata.json

```

{
  "version": 3,
  "artifactType": {
    "type": "APK",
    "kind": "Directory"
  },
  "applicationId": "com.example.namahatta",
  "variantName": "release",
  "elements": [
    {
      "type": "SINGLE",
      "filters": [],
      "attributes": [],
      "versionCode": 1,
      "versionName": "1.0",
      "outputFile": "app-release.apk"
    }
  ],
  "elementType": "File",

```

```

"baselineProfiles": [
  {
    "minApi": 28,
    "maxApi": 30,
    "baselineProfiles": [
      "baselineProfiles/1/app-release.dm"
    ]
  },
  {
    "minApi": 31,
    "maxApi": 2147483647,
    "baselineProfiles": [
      "baselineProfiles/0/app-release.dm"
    ]
  }
],
"minSdkVersionForDexing": 24
}

```

Название файла с расширением: src\androidTest\java\com\example\namahatta\ExampleInstrumentedTest.kt

```
package com.example.namahatta
```

```
import androidx.test.platform.app.InstrumentationRegistry
import androidx.test.ext.junit.runners.AndroidJUnit4
```

```
import org.junit.Test
import org.junit.runner.RunWith
```

```
import org.junit.Assert.*
```

```

/**
 * Instrumented test, which will execute on an Android device.
 *
 * See [testing documentation](http://d.android.com/tools/testing).
 */
@RunWith(AndroidJUnit4::class)
class ExampleInstrumentedTest {
    @Test
    fun useAppContext() {
        // Context of the app under test.
        val appContext = InstrumentationRegistry.getInstrumentation().targetContext
        assertEquals("com.example.namahatta", appContext.packageName)
    }
}

```

Название файла с расширением: src\main\AndroidManifest.xml

```

<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

```

```
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.WAKE_LOCK" />
```

```
<application
    android:allowBackup="true"
    android:dataExtractionRules="@xml/data_extraction_rules"
    android:fullBackupContent="@xml/backup_rules"
    android:icon="@mipmap/ic_launcher"
    android:label="@string/app_name"
    android:roundIcon="@mipmap/ic_launcher_round"
    android:supportsRtl="true"
    android:theme="@style/Theme.Namahatta">
```

```
<!-- Сервис FCM ДОЛЖЕН быть внутри <application> -->
```

```
<service
    android:name=".MyFirebaseMessagingService"
    android:exported="false">
    <intent-filter>
        <action android:name="com.google.firebase.MESSAGING_EVENT" />
    </intent-filter>
</service>
```

```
<activity
    android:name=".MainActivity"
    android:exported="true">
    <intent-filter>
        <action android:name="android.intent.action.MAIN" />
        <category android:name="android.intent.category.LAUNCHER" />
    </intent-filter>
</activity>
```

```
<activity
    android:name=".GroupApplicationsActivity"
    android:exported="true">
    <!-- Добавим, чтобы активити могла запускаться через уведомления -->
    <intent-filter>
        <action android:name="android.intent.action.VIEW" />
        <category android:name="android.intent.category.DEFAULT" />
    </intent-filter>
</activity>
```

```
<activity
    android:name=".EventDetailActivity"
    android:exported="false" />
```

```
<activity
    android:name=".GroupMembersActivity"
    android:exported="true" />
```

```
</application>
```

```
</manifest>
```

Название файла с расширением: src\main\java\com\example\namahatta\EventDetailActivity.kt

package com.example.namahatta // ← Должен совпадать с твоим package

```

import android.os.Bundle
import androidx.appcompat.app.AppCompatActivity

class EventDetailActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_event_detail)

        val eventId = intent.getStringExtra("event_id")
        if (eventId != null) {
            loadEvent(eventId)
        } else {
            finish() // Нет ID — закрываем активность
        }
    }

    private fun loadEvent(eventId: String) {
        // Здесь загрузишь данные события по ID
        // Например, через API или WebView
    }
}

```

Название файла с расширением: src\main\java\com\example\namahatta\GroupApplicationsActivity.kt

package com.example.namahatta // ← Должен совпадать с твоим package

```

import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.widget.Toast
import com.example.namahatta.GroupApplicationsFragment

class GroupApplicationsActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_group_applications)

        val groupId = intent.getStringExtra("group_id")
        if (groupId != null) {
            loadApplications(groupId)
        } else {
            Toast.makeText(this, "Ошибка: не указан ID группы", Toast.LENGTH_SHORT).show()
            finish()
        }
    }

    private fun loadApplications(groupId: String) {
        // Загружаем заявки для группы
        val fragment = GroupApplicationsFragment.newInstance(groupId)
        supportFragmentManager.beginTransaction()
            .replace(R.id.container, fragment)
            .commit()
    }
}

```

```
}
```

Название файла с расширением: src\main\java\com\example\namahatta\GroupApplicationsFragment.kt

package com.example.namahatta // ← Должен совпадать с твоим пакетом

```
import android.os.Bundle
import androidx.fragment.app.Fragment
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup

class GroupApplicationsFragment : Fragment() {

    private lateinit var groupId: String

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        arguments?.let {
            groupId = it.getString("group_id") ?: ""
        }
    }

    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        // Здесь будет логика загрузки заявок
        return inflater.inflate(R.layout.fragment_group_applications, container, false)
    }

    companion object {
        @JvmStatic
        fun newInstance(group_id: String) =
            GroupApplicationsFragment().apply {
                arguments = Bundle().apply {
                    putString("group_id", group_id)
                }
            }
    }
}
```

Название файла с расширением: src\main\java\com\example\namahatta\GroupMembersActivity.kt

package com.example.namahatta

```
import android.os.Bundle
import androidx.appcompat.app.AppCompatActivity
import android.webkit.WebView
import android.webkit.WebViewClient
import android.util.Log
```

```

class GroupMembersActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_group_members)

        val groupId = intent.getStringExtra("group_id")
        if (groupId != null) {
            loadGroupMembers(groupId)
        } else {
            Log.e("GroupMembers", "group_id не передан")
            finish()
        }
    }

    private fun loadGroupMembers(groupId: String) {
        val webView = findViewById<WebView>(R.id.webview)
        webView.webViewClient = WebViewClient()
        webView.settings.javaScriptEnabled = true
        webView.settings.domStorageEnabled = true

        // Загружаем админку с якорем или параметром для отображения участников
        webView.loadUrl("https://namahata.ru/admin.html#members-$groupId")
    }
}

```

Название файла с расширением: src\main\java\com\example\namahatta\MainActivity.kt

```
package com.example.namahatta
```

```

import android.content.Context
import android.os.Bundle
import android.webkit.JavascriptInterface
import android.webkit.WebView
import android.webkit.WebViewClient
import androidx.activity.OnBackPressedCallback
import androidx.activity.enableEdgeToEdge
import androidx.appcompat.app.AppCompatActivity
import androidx.core.view.ViewCompat
import androidx.core.view.WindowInsetsCompat
import com.google.firebase.FirebaseApp
import android.util.Log
import android.webkit.CookieManager
import android.webkit.WebResourceRequest
import com.google.firebase.messaging.FirebaseMessaging
import okhttp3.*
import okhttp3.RequestBody.Companion.toRequestBody
import okhttp3.MediaType.Companion.toMediaType
import org.json.JSONObject
import java.io.IOException

```

```
class MainActivity : AppCompatActivity() {
```

```
    private lateinit var webView: WebView
```

```

override fun onCreate(savedInstanceState: Bundle?) {
    Log.d("APP_DEBUG", "MainActivity started")
    super.onCreate(savedInstanceState)
    FirebaseApp.initializeApp(this)
    Log.d("APP_DEBUG", "Firebase initialized")

    // Получаем и сохраняем FCM-токен (но НЕ отправляем его)
    FirebaseMessaging.getInstance().token.addOnCompleteListener { task ->
        if (task.isSuccessful) {
            val token = task.result
            Log.d("FCM_TOKEN", "Получен токен: $token")

            // Сохраняем токен локально
            val prefs = getSharedPreferences("app_prefs", Context.MODE_PRIVATE)
            prefs.edit().putString("fcm_token", token).apply()

            // Попытка отправить токен, если уже известны user_id и PHPSESSID
            trySendTokenIfPossible()
        } else {
            Log.e("FCM_TOKEN", "Ошибка получения токена", task.exception)
        }
    }
}

enableEdgeToEdge()
setContentView(R.layout.activity_main)

webView = findViewById(R.id.webview)
val cookieManager = android.webkit.CookieManager.getInstance()
cookieManager.setAcceptCookie(true)
cookieManager.setAcceptThirdPartyCookies(webView, true)

webView.settings.apply {
    mixedContentMode = android.webkit.WebSettings.MIXED_CONTENT_ALWAYS_ALLOW
    domStorageEnabled = true
    javaScriptEnabled = true
    allowFileAccess = true
    allowContentAccess = true
    setGeolocationEnabled(false) // отключаем лишнее
}
webView.webViewClient = WebViewClient()
webView.settings.javaScriptEnabled = true
webView.settings.domStorageEnabled = true
webView.addJavascriptInterface(WebAppInterface(this), "Android")

// :fire: Загружаем URL ОДИН РАЗ
val loadUrl = intent.getStringExtra("load_url")
if (loadUrl != null) {
    webView.loadUrl(loadUrl)
} else {
    webView.loadUrl("https://namahata.ru/index.html")
}

onBackPressedDispatcher.addCallback(this, object : OnBackPressedCallback(true) {

```

```

override fun handleOnBackPressed() {
    val currentUrl = webView.url ?: ""

    // Если мы на странице входа/регистрации — идём на главную
    if (currentUrl.contains("login") || currentUrl.contains("auth") || currentUrl.contains("register")) {
        webView.loadUrl("https://namahata.ru/index.html")
        return
    }
    if (webView.canGoBack()) {
        webView.goBack()
    } else {
        isEnabled = false
        onBackPressedDispatcher.onBackPressed()
    }
}
})

ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main)) { v, insets ->
    val systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars())
    v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom)
    insets
}
}

private fun sendTokenToServer(token: String, sessionId: String) {
    val json = JSONObject().apply {
        put("fcm_token", token)
    }.toString()

    val client = OkHttpClient()
    val mediaType = "application/json; charset=utf-8".toMediaType()
    val requestBody = json.toRequestBody(mediaType)
    val request = Request.Builder()
        .url("https://namahata.ru/api/save_fcm_token.php")
        .addHeader("Cookie", "PHPSESSID=$sessionId")
        .post(requestBody)
        .build()

    client.newCall(request).enqueue(object : Callback {
        override fun onFailure(call: Call, e: IOException) {
            Log.e("FCM_SEND", "Ошибка отправки: ${e.message}", e)
        }

        override fun onResponse(call: Call, response: Response) {
            val result = response.body?.string() ?: "No response body"
            if (response.isSuccessful) {
                Log.d("FCM_SEND", "Токен успешно отправлен")
            } else {
                Log.e("FCM_SEND", "Ошибка сервера: ${response.code} - $result")
            }
        }
    })
}
}

```

```

private fun trySendTokenIfPossible() {
    val prefs = getSharedPreferences("app_prefs", Context.MODE_PRIVATE)
    val fcmToken = prefs.getString("fcm_token", null)
    val userId = prefs.getString("user_id", null)

    if (fcmToken != null && userId != null) {
        // Получаем PHPSESSID из WebView
        val cookieManager = CookieManager.getInstance()
        val cookies = cookieManager.getCookie("https://namahata.ru")
        var phpSessionId: String? = null

        cookies?.split(";")?.forEach { cookie ->
            val parts = cookie.trim().split("=", limit = 2)
            if (parts.size == 2 && parts[0] == "PHPSESSID") {
                phpSessionId = parts[1]
            }
        }

        if (phpSessionId != null) {
            sendTokenToServer(fcmToken, phpSessionId)
        }
    }
}

// :fire: ДОБАВЬ inner ЗДЕСЬ:
inner class WebAppInterface(private val context: Context) {
    @JavascriptInterface
    fun setUser(userId: String) {
        val prefs = context.getSharedPreferences("app_prefs", Context.MODE_PRIVATE)
        prefs.edit().putString("user_id", userId).apply()
        Log.d("WEB_APP", "user_id установлен: $userId")

        // Попробуем отправить токен, если всё готово
        if (context is MainActivity) {
            context.trySendTokenIfPossible()
        } else {
            Log.w("WEB_APP", "Контекст не является MainActivity — пропуск отправки токена")
        }
    }
}
}

```

Название файла с расширением: src\main\java\com\example\namahatta\MyFirebaseMessagingService.kt

```
package com.example.namahatta
```

```

import android.app.NotificationChannel
import android.app.NotificationManager
import android.content.Context
import android.content.Intent
import android.util.Log
import androidx.core.content.edit
import com.google.firebase.messaging.FirebaseMessagingService

```

```
import com.google.firebase.messaging.RemoteMessage
import okhttp3.*
import org.json.JSONObject
import java.io.IOException
import android.content.SharedPreferences
import okhttp3.MediaType
import okhttp3.MediaType.Companion.toMediaType
import okhttp3.RequestBody.Companion.toRequestBody
```

```
import android.app.PendingIntent
import androidx.core.app.NotificationCompat
import com.example.namahatta.MainActivity
```

```
import android.os.Build
// Обязательно проверьте, что R импортирован из вашего пакета:
import com.example.namahatta.R
```

```
class MyFirebaseMessagingService : FirebaseMessagingService() {
```

```
    private val TAG = "MyFirebaseMsgService"
```

```
    override fun onMessageReceived(remoteMessage: RemoteMessage) {
```

```
        val data = remoteMessage.data
```

```
        val action = data["action"] ?: "default"
```

```
        val groupId = data["group_id"]
```

```
        // Создаем интент для перехода в нужное место
```

```
        val intent = when (action) {
```

```
            "view_group_applications" -> {
```

```
                Intent(this, MainActivity::class.java).apply {
```

```
                    // Передаём URL с якорем или параметром
```

```
                    putExtra("load_url", "https://namahata.ru/admin.html?group_id=${data["group_id"]}&tab=applications")
```

```
                }
```

```
            }
```

```
            "view_group_members" -> {
```

```
                Intent(this, MainActivity::class.java).apply {
```

```
                    putExtra("load_url", "https://namahata.ru/admin.html?group_id=${data["group_id"]}&tab=members")
```

```
                }
```

```
            }
```

```
            "view_event" -> {
```

```
                Intent(this, MainActivity::class.java).apply {
```

```
                    putExtra("load_url", "https://namahata.ru/event.html?id=${data["event_id"]}")
```

```
                }
```

```
            }
```

```
            else -> Intent(this, MainActivity::class.java)
```

```
        }
```

```
        // Делаем интент "одноразовым"
```

```
        intent.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_CLEAR_TOP)
```

```
        val pendingIntent = PendingIntent.getActivity(
```

```
            this,
```

```
            System.currentTimeMillis().toInt(), // Уникальный ID
```

```

        intent,
        PendingIntent.FLAG_IMMUTABLE or PendingIntent.FLAG_UPDATE_CURRENT
    )

    showNotification(
        remoteMessage.notification?.title ?: "Новое уведомление",
        remoteMessage.notification?.body ?: "Проверьте уведомления",
        pendingIntent
    )
}

// Добавляем эту функцию в класс
private fun showNotification(title: String, message: String, pendingIntent: PendingIntent) {
    val channelId = "default_channel"
    val manager = getSystemService(Context.NOTIFICATION_SERVICE) as NotificationManager

    // Создаём канал для Android 8.0+
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
        val channel = NotificationChannel(
            channelId,
            "Уведомления",
            NotificationManager.IMPORTANCE_HIGH
        ).apply {
            description = "Канал для важных уведомлений"
        }
        manager.createNotificationChannel(channel)
    }

    // Строим уведомление
    val builder = NotificationCompat.Builder(this, channelId)
        .setSmallIcon(android.R.drawable.ic_dialog_info) // ← Временная иконка
        .setContentTitle(title)
        .setContentText(message)
        .setPriority(NotificationCompat.PRIORITY_HIGH)
        .setContentIntent(pendingIntent)
        .setAutoCancel(true)

    // Отправляем уведомление с уникальным ID
    manager.notify(System.currentTimeMillis().toInt(), builder.build())
}

override fun onNewToken(token: String) {
    Log.d(TAG, "Новый FCM-токен: $token")
    sendTokenToServer(token)
}

private fun sendTokenToServer(token: String) {
    // Получаем user_id из SharedPreferences
    val prefs: SharedPreferences = getSharedPreferences("app_prefs", Context.MODE_PRIVATE)
    val userId = prefs.getString("user_id", null)

    if (userId == null) {
        Log.w(TAG, "user_id не найден, пропускаем отправку токена")
        return
    }
}

```

```

}

// Подготавливаем JSON
val json = JSONObject().apply {
    put("user_id", userId)
    put("fcm_token", token)
}.toString()

// Настройка OkHttpClient
val client = OkHttpClient()
val mediaType = "application/json; charset=utf-8".toMediaType()
val requestBody = json.toRequestBody(mediaType)
val request = Request.Builder()
    .url("https://namahata.ru/api/save_fcm_token.php")
    .post(requestBody)
    .build()

// Асинхронная отправка
client.newCall(request).enqueue(object : Callback {
    override fun onFailure(call: Call, e: IOException) {
        Log.e(TAG, "Ошибка отправки FCM-токена: ${e.message}", e)
    }

    override fun onResponse(call: Call, response: Response) {
        if (response.isSuccessful) {
            Log.d(TAG, "FCM-токен успешно отправлен на сервер")
        } else {
            Log.e(TAG, "Ошибка сервера: ${response.code} - ${response.body?.string()}")
        }
    }
})
}
}
}

```

Название файла с расширением: src\main\res\drawable\ic_launcher_background.xml

```

<?xml version="1.0" encoding="utf-8"?>
<vector
    android:height="108dp"
    android:width="108dp"
    android:viewportHeight="108"
    android:viewportWidth="108"
    xmlns:android="http://schemas.android.com/apk/res/android">
    <path android:fillColor="#3DDC84"
        android:pathData="M0,0h108v108h-108z"/>
    <path android:fillColor="#00000000" android:pathData="M9,0L9,108"
        android:strokeColor="#33FFFFFF" android:strokeWidth="0.8"/>
    <path android:fillColor="#00000000" android:pathData="M19,0L19,108"
        android:strokeColor="#33FFFFFF" android:strokeWidth="0.8"/>
    <path android:fillColor="#00000000" android:pathData="M29,0L29,108"
        android:strokeColor="#33FFFFFF" android:strokeWidth="0.8"/>
    <path android:fillColor="#00000000" android:pathData="M39,0L39,108"
        android:strokeColor="#33FFFFFF" android:strokeWidth="0.8"/>

```

[illegible]

```
<path android:fillColor="#00000000" android:pathData="M79,19L79,89"
      android:strokeColor="#33FFFFFF" android:strokeWidth="0.8"/>
</vector>
```

Название файла с расширением: src\main\res\drawable\ic_launcher_foreground.xml

```
<vector xmlns:android="http://schemas.android.com/apk/res/android"
      xmlns:aapt="http://schemas.android.com/aapt"
      android:width="108dp"
      android:height="108dp"
      android:viewportWidth="108"
      android:viewportHeight="108">
    <path android:pathData="M31,63.928c0,0 6.4,-11 12.1,-13.1c7.2,-2.6 26,-1.4
26,-1.4l38.1,38.1L107,108.928l-32,-1L31,63.928z">
      <aapt:attr name="android:fillColor">
        <gradient
          android:endX="85.84757"
          android:endY="92.4963"
          android:startX="42.9492"
          android:startY="49.59793"
          android:type="linear">
            <item
              android:color="#44000000"
              android:offset="0.0" />
            <item
              android:color="#00000000"
              android:offset="1.0" />
          </gradient>
        </aapt:attr>
      </path>
      <path
        android:fillColor="#FFFFFF"
        android:fillType="nonZero"
        android:pathData="M65.3,45.828l3.8,-6.6c0.2,-0.4 0.1,-0.9 -0.3,-1.1c-0.4,-0.2 -0.9,-0.1 -1.1,0.3l-3.9,6.7c-6.3,-2.8
-13.4,-2.8 -19.7,0l-3.9,-6.7c-0.2,-0.4 -0.7,-0.5 -1.1,-0.3C38.8,38.328 38.7,38.828 38.9,39.228l3.8,6.6C36.2,49.428
31.7,56.028 31,63.928h46C76.3,56.028 71.8,49.428 65.3,45.828zM43.4,57.328c-0.8,0 -1.5,-0.5 -1.8,-1.2c-0.3,-0.7
-0.1,-1.5 0.4,-2.1c0.5,-0.5 1.4,-0.7 2.1,-0.4c0.7,0.3 1.2,1 1.2,1.8C45.3,56.528 44.5,57.328
43.4,57.328L43.4,57.328zM64.6,57.328c-0.8,0 -1.5,-0.5 -1.8,-1.2s-0.1,-1.5 0.4,-2.1c0.5,-0.5 1.4,-0.7 2.1,-0.4c0.7,0.3
1.2,1 1.2,1.8C66.5,56.528 65.6,57.328 64.6,57.328L64.6,57.328z"
        android:strokeWidth="1"
        android:strokeColor="#00000000" />
      </path>
    </vector>
```

Название файла с расширением: src\main\res\layout\activity_event_detail.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
      android:layout_width="match_parent"
      android:layout_height="match_parent"
      android:orientation="vertical"
      android:padding="16dp">
```

```
<TextView
    android:id="@+id/tvEventTitle"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Загрузка события..."
    android:textSize="18sp" />
```

```
</LinearLayout>
```

Название файла с расширением: src\main\res\layout\activity_group_applications.xml

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/container"
    android:layout_width="match_parent"
    android:layout_height="match_parent" />
```

Название файла с расширением: src\main\res\layout\activity_group_members.xml

```
<?xml version="1.0" encoding="utf-8"?>
<WebView xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/webview"
    android:layout_width="match_parent"
    android:layout_height="match_parent" />
```

Название файла с расширением: src\main\res\layout\activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <WebView
        android:id="@+id/webview"
        android:layout_width="0dp"
        android:layout_height="0dp"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

Название файла с расширением: src\main\res\layout\fragment_group_applications.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">

    <TextView
        android:id="@+id/tvGroupInfo"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Загрузка заявок..." />

</LinearLayout>
```

Название файла с расширением: src\main\res\mipmap-anydpi-v26\ic_launcher.xml

```
<?xml version="1.0" encoding="utf-8"?>
<adaptive-icon xmlns:android="http://schemas.android.com/apk/res/android">
    <background android:drawable="@drawable/ic_launcher_background"/>
    <foreground android:drawable="@mipmap/ic_launcher_foreground"/>
</adaptive-icon>
```

Название файла с расширением: src\main\res\mipmap-anydpi-v26\ic_launcher_round.xml

```
<?xml version="1.0" encoding="utf-8"?>
<adaptive-icon xmlns:android="http://schemas.android.com/apk/res/android">
    <background android:drawable="@drawable/ic_launcher_background"/>
    <foreground android:drawable="@mipmap/ic_launcher_foreground"/>
</adaptive-icon>
```

Название файла с расширением: src\main\res\values\colors.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="black">#FF000000</color>
    <color name="white">#FFFFFFFF</color>
</resources>
```

Название файла с расширением: src\main\res\values\strings.xml

```
<resources>
    <string name="app_name">namahatta</string>
</resources>
```

Название файла с расширением: src\main\res\values\themes.xml

```
<resources xmlns:tools="http://schemas.android.com/tools">
    <!-- Base application theme. -->
```

```

<style name="Base.Theme.Namahatta" parent="Theme.Material3.DayNight.NoActionBar">
    <!-- Customize your light theme here. -->
    <!-- <item name="colorPrimary">@color/my_light_primary</item> -->
</style>

<style name="Theme.Namahatta" parent="Base.Theme.Namahatta" />
</resources>

```

Название файла с расширением: src\main\res\values-night\themes.xml

```

<resources xmlns:tools="http://schemas.android.com/tools">
    <!-- Base application theme. -->
    <style name="Base.Theme.Namahatta" parent="Theme.Material3.DayNight.NoActionBar">
        <!-- Customize your dark theme here. -->
        <!-- <item name="colorPrimary">@color/my_dark_primary</item> -->
    </style>
</resources>

```

Название файла с расширением: src\main\res\xml\backup_rules.xml

```

<?xml version="1.0" encoding="utf-8"?><!--
Sample backup rules file; uncomment and customize as necessary.
See https://developer.android.com/guide/topics/data/autobackup
for details.
Note: This file is ignored for devices older than API 31
See https://developer.android.com/about/versions/12/backup-restore
-->
<full-backup-content>
    <!--
    <include domain="sharedpref" path="."/>
    <exclude domain="sharedpref" path="device.xml"/>
    -->
</full-backup-content>

```

Название файла с расширением: src\main\res\xml\data_extraction_rules.xml

```

<?xml version="1.0" encoding="utf-8"?><!--
Sample data extraction rules file; uncomment and customize as necessary.
See https://developer.android.com/about/versions/12/backup-restore#xml-changes
for details.
-->
<data-extraction-rules>
    <cloud-backup>
        <!-- TODO: Use <include> and <exclude> to control what is backed up.
        <include .../>
        <exclude .../>
        -->
    </cloud-backup>
    <!--
    <device-transfer>
        <include .../>

```

```
<exclude .../>
</device-transfer>
-->
</data-extraction-rules>
```

Название файла с расширением: src\test\java\com\example\namahatta\ExampleUnitTest.kt

```
package com.example.namahatta
```

```
import org.junit.Test
```

```
import org.junit.Assert.*
```

```
/**
 * Example local unit test, which will execute on the development machine (host).
 *
 * See [testing documentation](http://d.android.com/tools/testing).
 */
class ExampleUnitTest {
    @Test
    fun addition_isCorrect() {
        assertEquals(4, 2 + 2)
    }
}
```
